



Recommender System for Amazon



1. Introduction

Recommender systems have become a very important part of browsing content online. From entertainment to shopping, the quantity of unique items is too vast to explore one at a time. Recommender systems can suggest items or content that is relevant to an individual to help them find items of interest with little effort. The motivation for our decision comes from how commonly these are used and how effective they can be.

We were initially interested in building a system that can make recommendations based on a user's online behaviour. We noticed, whilst searching, that the vast majority of recommender systems available are focused on one specific domain, such as movies, books etc. We decided that we would try and build a system that can make accurate recommendations across domains, recommending books to users based on the books/video games they consume. We decided to use a dataset of [Amazon Customer Reviews](#) on a collection of different kinds of products to achieve this.

To keep our implementation as accurate to the real world use case as possible we decided to take an approach that uses Spark. This data would be collected continually and enter our system through a stream, so we should mimic this input to keep the use case in mind. We will also make use of several components of the Hadoop framework to pre-process and query our data, which will likely be of a high volume. These decisions with regard to the technology used should allow us to create a model that can be applied to the real use case.

SparkML offers an implementation of an alternating least squares (ALS) matrix factorization for collaborative filtering models. While this implementation is good, it runs on a batch of data, so does not directly support streaming. This means that to create a model for processing a data stream we will need to take the latent representation of the user and item matrices and create functions to update these vectors based on new observations. This model will not be making use of any side features as these are not available within our data and this ALS implementation does not work well with side features.

2. What Data

The [Amazon US Customer Reviews Dataset](#) contains countless reviews for products ranging across many different categories. These reviews were collected solely in the US and contain several fields such as IDs for the customer, product and review, the product name, and the rating, review title and content just to name several features. Many of these features will not be of use as we will be looking at a collaborative filtering approach, so the ratings are the key feature. We have decided to look only at a subset of these that we can consider similar, this subset being books, video DVDs and video games. This decision was made based on the hope that there will be overlap in customers as all categories cover entertainment in different mediums.

Specifically, we chose this subset as we thought that it was highly likely that people who bought and reviewed movies would also have bought and reviewed books and videogames.

3. What Analytics

The task of preparing the data was a significant one, due to the initial size of the datasets. A quick overview of the data revealed that there were large amounts of rows that had many empty values.

For two reasons, we opted to remove all of the rows that had null values:

- 1) Each row had different missing values. It was not feasible for us to identify which values needed to be filled for every row in each dataset.

- 2) There were null values in columns such as review_id and customer_id. It was impossible to impute these values as they are unique identifiers.

We also dropped the following columns from the initial dataset.

- vine
- verified_purchase
- review_headline
- review_body
- Review_date

These tasks were performed in Apache Pig and the data was saved in multiple TSV files. We had four other main preprocessing tasks that we had to perform to be able to train our ALS model on the data that we had.

- **Concatenate Tables**
 - We had to combine each one of the various tables to make one large dataset of all the reviews.
- **Remove Columns**
 - As they were not needed for the recommendation task, we removed the marketplace, review_id, product_parent, helpful_votes and total_votes columns
- **Product ID**
 - The PySpark ALS model requires the 'User' column (productID in our case) to be numeric only. Our productID was alphanumeric. We had to assign a new identifier value to each product.
- **Rating**
 - We had to normalise the ratings as there are some users that would leave harsher ratings than others.

4. Analytics task and results

Our main task with this project was to create a collaborative filtering recommendation system on the aforementioned Amazon customer review data. Our goal was to allow for recommendations between different kinds of products. In other words, to be able to recommend a movie to someone based on what books they had reviewed positively, even if they had not reviewed any movies up to that point.

We thus opted to use a collaborative filtering method for our recommendation system, given that we felt that like-minded people would have similar tastes in books, movies and video games. If we had attempted to produce a content-based filtering system, it would have produced poorer results as we found it unlikely that a user would for similar content in a video game as they would in a movie.

Our initial matrix, constructed after the steps above in the Data Cleaning stage, was highly sparse. This is due to a high number of products compared to ratings, leading to a high amount of products with few ratings. To address this, we needed to use matrix factorisation.

This will help by:

1. Letting the model learn to produce better, more personal recommendations for users by factorising the original matrix in user and product matrices.
2. Allowing the model to recommend suitable lesser-known products, rather than well-known, less suitable products.

We decided to use the ALS model from the pyspark.ml.recommendation library as it is built for this kind of large-scale collaborative filtering task.#

We used Streamlit to make an interactive UI to go along with our recommender system so that a user could navigate their recommendations in a more intuitive way.

5. Related Work – any similar analysis used in any known application?

Netflix uses matrix factorisation for its own recommender systems, along with the use of other machine learning techniques. [1]

Until recently, YouTube implicitly used matrix factorisation for its recommender systems until recently, where they adopted a deep-learning approach. [2]

From these examples, it's clear that ALS and matrix factorisation are and can be used to tackle similar large-scale recommendation system problems.

6. Challenges and lessons learned from working on the selected dataset with the selected technology

We had to deal with quite a few challenges with our dataset and technology

- Our movie review data had over 4 million rows. We did not have the computing power available to use to complete the Pig tasks effectively, when ingesting all the movies data in one go. We split the dataset in half and ran the same script on each half and merged the results.
- Initially, our recommender system produced poor results. Our first test run produced a RMSE of ~4.3. Given that the rating is out of 5, this is arguably worse than guessing. However, when we assigned new, numeric values to productID and normalised the ratings, we achieved a RMSE of ~0.44.

7. Responsibility statement (which group member did what?)

Belbin Team Roles:

Resource Investigator - 

Plant 

Monitor Evaluator - 

Specialist 

Shaper - 

Implementer - 

Completer Finisher - 

Specific Tasks



Ran Hive queries for EDA.

Additional data cleaning and visualisations/
fine-tuning and improving ALS model.

Built Streamlit interactive UI



Ran Pig queries for data cleaning.

Built raw, initial ALS model

Put together final report.

Distribution of marks It was decided unanimously that both members of the team be rewarded 50% of the available marks, i.e half of the total mark each.

8. Response to peer feedback

The feedback we received was helpful in directing us in the best possible way to use the data that we had in the best extent to make our recommender system as accurate as possible.

Our feedback said that our data was probably not sufficient to make a content-based recommendation system which confirmed our initial thought that a collaborative filtering model would be the best for us. We were not aware of any API's that could give us more information on each product given the productID and even if that

was possible, it would have been very compute-intensive and time consuming to do that for our millions of reviews.

References

- [1] Redaktor. "Netflix Recommendation System: How It Works - RecoAI." *RecoAI*, 5 Apr. 2022, recoai.net/netflix-recommendation-system-how-it-works/. Accessed 14 Dec. 2022.
- [2] Nadeem, Moin. "How YouTube Recommends Videos - towards Data Science." *Medium*, Towards Data Science, 7 July 2018, towardsdatascience.com/how-youtube-recommends-videos-b6e003a5ab2f. Accessed 14 Dec. 2022.